

Theory of Computation @ HNU CE (2026년 2학기)

kya

2026-09-23

차례

HW02 NFA로 문자열 인식	3
주교재 Figure 1.30	3
주교재 Figure 1.27	4
주교재 Figure 1.27 + $q_2 \xleftarrow{\epsilon} q_3$	5
1. Prolog 소개	6
1.1. 가족 관계 예시	6
1.2. 리스트 기초, 출력을 위한 술어, 연결자(connective) 우선순위	7
1.3. 리스트 연결 예시	9
1.4. 사실(fact), 규칙(rule), 절(clause), 술어(predicate)	11
1.5. 항(term)과 일치화(unification)	13
1.5.1. 항(term)	13
1.5.2. 일치화(unification)	14
주교재 0 Introduction	16
주교재 0.1 오토마타, 계산가능성, 복잡도	16
Complexity Theory 복잡도 이론	17
Computability Theory 계산가능성 이론	17
Automata Theory 오토마타 이론	17
주교재 0.2 수학적 표기법과 용어	18
집합 set	18
함수와 관계 functions and relations	20
관계의 표기법	22
그래프 Graph	23
문자열과 언어	24
주교재 1 정규언어 Regular Languages	25
주교재 1.1 유한 오토마타 Finite Automata	25
주교재 1.2 비결정성 Non-determinism	36
주교재 1.3 정규식 Regular Expressions	39
주교재 1.3 정규언어가 아닌 언어	39
다음 장	40

HW02 NFA로 문자열 인식

- 이름:
- 학번:

주교재 Figure 1.30

ϵ 이 없는 NFA에서 문자열 인식

```
1  ?- atom_chars('abac', CS), atom_chars(Str,  
    ['a','b','c']),  
2      format('atom_chars 실행 예시 ~w ~w ~n', [CS, Str]).  
3  
4  state([q1,q2,q3,q4]). % all states  
5  
6  % alphabet 정의는 생략  
7  
8  delta(q1, '0', q1). delta(q1, '1', q1). delta(q1, '1', q2).  
9  delta(q2, '0', q3). delta(q2, '1', q3).  
10 delta(q3, '0', q4). delta(q3, '1', q4).  
11  
12 start(q1). % start state  
13 final([q4]). % accept states  
14  
15 % Str은 '01011' 같은 atom 형태로 제공  
16 recognize(Str) :- start(Q), atom_chars(Str, CS, Q).  
17  
18 recog_chars([], Q) :- true. % true 대신 여기에 코드 작성.  
19 recog_chars([C|CS], Q) :- true. % true 대신 여기에 코드 작성.  
20  
21 ?- true, % 이 줄의 true 대신 적절한 첫째 쿼리 작성  
22     format('~w ~w!~n', ['1st','result']). % 적절한 형식으로 결과 출력  
23 ?- true, % 이 줄의 true 대신 적절한 첫째 쿼리 작성  
24     format('~w ~w!~n', ['2nd','result']). % 적절한 형식으로 결과 출력
```

0: atom_chars 실행 예시 [a,b,a,c] abc

E: Warning: /home/kyagr/grd/github/hnu-pl/TC2026/_build/TC26hw02Fig1dot30.pl:16:

E: Warning: Singleton variables: [CS]

E: Warning: /home/kyagr/grd/github/hnu-pl/TC2026/_build/TC26hw02Fig1dot30.pl:18:

E: Warning: Singleton variables: [Q]

E: Warning: /home/kyagr/grd/github/hnu-pl/TC2026/_build/TC26hw02Fig1dot30.pl:19:

0: 1st result!

0: 2nd result!

0:

E: Warning: Singleton variables: [C,CS,Q]

- 이름:
- 학번:

주교재 Figure 1.27

ϵ 이 있는 NFA에서 문자열 인식

```

1 state([q1,q2,q3,q4]). % all states
2
3 % alphabet 정의는 생략
4
5 delta(q1, '0', q1). delta(q1, '1', q1). delta(q1, '1', q2).
6 delta(q2, '0', q3). delta(q2, '', q3). % epsilon을 ''로 표현
7 delta(q3, '0', q4).
8 delta(q4, '0', q4). delta(q4, '1', q4).
9
10 start(q1). % start state
11 final([q4]). % accept states
12
13 % Str은 '01011' 같은 atom 형태로 제공
14 % Str은 '01011' 같은 atom 형태로 제공
15 recognize(Str) :- start(Q), atom_chars(Str, CS, Q).
16
17 recog_chars(CS, Q) :- true. % true 대신 여기에 코드 작성.
18 % 여러 개의 clause로 나눠서 작성해도 됨.
19 % 도우미/보조 predicate을 추가로 작성해
20 % 도 됨.
21
22 ?- true, % 이 줄의 true 대신 적절한 첫째 쿼리 작성
23 format('~w ~w!\n', ['1st','result']). % 적절한 형식으로 결과 출력
24 ?- true, % 이 줄의 true 대신 적절한 첫째 쿼리 작성
25 format('~w ~w!\n', ['2nd','result']). % 적절한 형식으로 결과 출력

```

```

E: Warning: /home/kyagr/github/hnu-pl/TC2026/_build/TC26hw02Fig1dot27.pl:15:
0: 1st result!
0: 2nd result!
0:
E: Warning: Singleton variables: [CS]
E: Warning: /home/kyagr/github/hnu-pl/TC2026/_build/TC26hw02Fig1dot27.pl:17:
E: Warning: Singleton variables: [CS,Q]

```

- 이름:
- 학번:

주교재 Figure 1.27 + $q_2 \xrightarrow{\epsilon} q_3$

ϵ 전이들로 이루어진 cycle이 있는 NFA에서 문자열 인식

```

1 state([q1,q2,q3,q4]). % all states
2
3 % alphabet 정의는 생략
4
5 delta(q1, '0', q1). delta(q1, '1', q1). delta(q1, '1', q2).
6 delta(q2, '0', q3). delta(q2, '', q3). % epsilon을 ''로 표현
7 delta(q3, '0', q4). delta(q3, '', q2).
8 delta(q4, '0', q4). delta(q4, '1', q4).
9
10 start(q1). % start state
11 final([q4]). % accept states
12
13 % Str은 '01011' 같은 atom 형태로 제공
14 % Str은 '01011' 같은 atom 형태로 제공
15 recognize(Str) :- start(Q), atom_chars(Str, CS, Q).
16
17 recog_chars(CS, Q) :- true. % true 대신 여기에 코드 작성.
18 % 여러 개의 clause로 나눠서 작성해도 됨.
19 % 도우미/보조 predicate을 추가로 작성해
20 % 도 됨.
21
22 ?- true, % 이 줄의 true 대신 적절한 첫째 쿼리 작성
23 format('~w ~w!\n', ['1st','result']). % 적절한 형식으로 결과 출력
24 ?- true, % 이 줄의 true 대신 적절한 첫째 쿼리 작성
25 format('~w ~w!\n', ['2nd','result']). % 적절한 형식으로 결과 출력

```

E: Warning: /home/kyagrdr/github/hnu-pl/TC2026/_build/TC26hw02epscyc.pl:15:

E: Warning: Singleton variables: [CS]

E: Warning: /home/kyagrdr/github/hnu-pl/TC2026/_build/TC26hw02epscyc.pl:17:

O: 1st result!

O: 2nd result!

O:

E: Warning: Singleton variables: [CS,Q]

1. Prolog 소개

Prolog 코드에서 %로 시작해 줄 끝까지 부분은 주석(comment)으로 취급됨.

1.1. 가족 관계 예시

```
1 % 1항 관계 male 원소나열법 표현
2 male(bob). male(john). % 원래 한줄에 하나씩 작성을 권장
3 % 1항 관계 female 원소나열법 표현
4 female(alice). female(carol). % 원래 한줄에 하나씩 작성을 권장
5 % 2항 관계 child 원소나열법 표현
6 child(bob, alice). child(carol, alice).
7 child(bob, john). child(carol, john).
8 % 2항 관계 son 조건제시법 표현
9 son(X, Y) :- child(X, Y), male(X).
10
11 ?- findall( (X,Y), son(X, Y) , Bag ),
12 forall( member((X,Y), Bag),
13 format("son: X = ~w, Y = ~w~n", [X, Y]) ).
```

 family.pl

0: son: X = bob, Y = alice

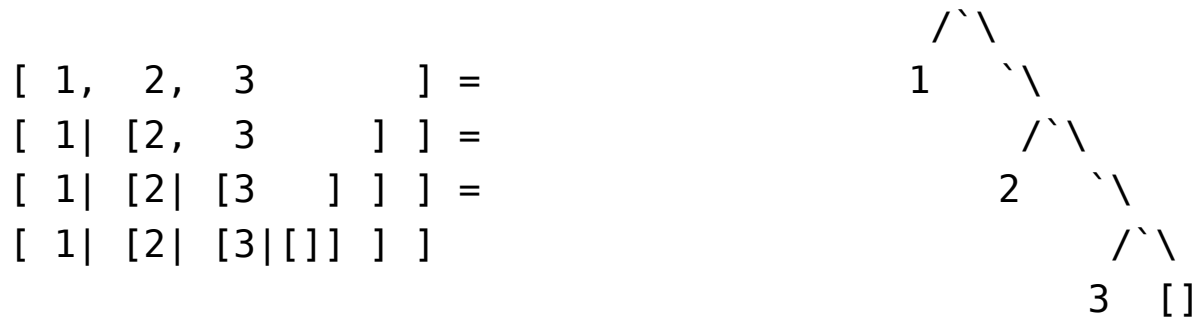
0: son: X = bob, Y = john

0:

1.2. 리스트 기초, 출력을 위한 술어, 연결자(connective) 우선순위

Prolog에서 리스트(list)는

- 길이 0인 빈 리스트는 `[]`로 표현
- `[1,2,3]` 처럼 쉼표(`,`)로 구분하여 나열한 원소를 대괄호(`[`와 `]`)로 감싸서 표현
- `[H|T]` 처럼 하나의 원소(`H`, 머리)와 나머지 리스트(`T`, 꼬리)로 나누어 표현



실제로 `write` 술어(predicate)로 출력해 보면 똑같다.

```
1 ?- L = [ 1, 2, 3 ], write(L), nl.  
2 ?- L = [ 1| [2, 3 ] ], write(L), nl.  
3 ?- L = [ 1| [2| [3 ] ] ], write(L), nl.  
4 ?- L = [ 1| [2| [3|[]] ] ], write(L), nl.
```

 **listbasic.pl**

```
0: [1,2,3]  
0: [1,2,3]  
0: [1,2,3]  
0: [1,2,3]
```

Prolog에서는 조건 `->` 참인_경우 ; 거짓인_경우 형태로 if-then-else를 표현 가능

```
6 ?- [1,2,3] = [1,[2,3]] -> (write(true), nl) ; (write(false), nl).  
7 ?- [1,2,3] = [1|[2,3]] -> (write(true), nl) ; (write(false), nl).
```

 listbasic.pl

0: false

0: true

and 연결자(`,`)의 우선순위가 if 연결자(`->`)와 or 연결자(`;`)보다 높으므로 괄호 생략 가능

```
9 ?- [1,2,3] = [1,[2,3]] -> write(true), nl ; write(false), nl .  
10 ?- [1,2,3] = [1|[2,3]] -> write(true), nl ; write(false), nl .
```

 listbasic.pl

0: false

0: true

하나의 파일 `listbasic.pl`을 이렇게 여러 부분의 코드 조각으로 나누어 작성하는 것도 가능하다.

좀 더 복잡한 형식의 출력은 C의 `printf`와 비슷한 `format` 술어를 활용

```
12 ?- L1 = [1], L2 = [2,3], format("L1 = ~w, L2 = ~w", [L1,L2]).
```

 listbasic.pl

0: L1 = [1], L2 = [2,3]

0:

1.3. 리스트 연결 예시

`app(L1,L2,L)` 은 두 리스트 `L1` 과 `L2` 를 이어붙이면 `L` 이 되는 3항 관계

```
1 app([], L, L). % 빈 리스트와 L을 연결하면 L
2 app([H|L1], L2, [H|L]) :- app(L1, L2, L). % 길이 1이상의 리스트와 L2 연결하는 재귀적 규칙
3
4 ?- app([1], [2,3], L), % 관계를 만족하는 L을 탐색
5     format("L = ~w~n", [L]). % L을 적절한 형식으로 출력
```

 listapp.pl

0: L = [1,2,3]

원래 `app` 을 정의할 때 생각했던 것과 반대 방향으로도 활용 가능!

- `L1` 과 `L2` 를 연결해 `L` 이 되는 관계를 생각하고 정의했는데 ...
- 반대로 `L` 을 `L1` 과 `L2` 로 나누는 방법도 탐색 가능!!!

(참고로, 이런 반대 방향 탐색이 모든 정의에 대해 항상 잘 되는 것은 아님)

```
7 ?- app(L1, L2, [1,2,3]), % 관계를 만족하는 1가지 경우를 찾아서
8     format("L1 = ~w, L2 = ~w~n", [L1, L2]). % 적절한 형식으로 출력
```

 listapp.pl

0: L1 = [], L2 = [1,2,3]

길이 0인 `L1` 인 당연한 경우만 탐색되어 실망했다면 ... 다음 페이지를 보라

`app(L1,L2,[1,2,3])` 를 만족하는 모든 경우를 탐색하려면 `findall` 을 활용해 질의(query)

```
10 ?- forall( (L1,L2), app(L1, L2, [1,2,3]), Bag ), % 모든 경우의 집합 Bag 
11     forall( member((L1,L2), Bag), % 모든 경우를 순회하며 Bag의 원소 [L1,L2]를 하나씩
12         format("L1 = ~w, L2 = ~w~n", [L1, L2]) ). % 적절한 형식으로 출력
```

0: L1 = [], L2 = [1,2,3]

0: L1 = [1], L2 = [2,3]

0: L1 = [1,2], L2 = [3]

0: L1 = [1,2,3], L2 = []

최대 몇 개까지만 찾으려면 `findsols` 를 활용해 질의(query)

```
14 ?- findnsols( 2, % 최대 2개까지만 만족하는 해를 찾기 
15     (L1,L2), app(L1, L2, [1,2,3]), Bag ),
16     forall( member((L1,L2), Bag),
17         format("L1 = ~w, L2 = ~w~n", [L1, L2]) ).
```

0: L1 = [], L2 = [1,2,3]

0: L1 = [1], L2 = [2,3]

0:

1.4. 사실(fact), 규칙(rule), 절(clause), 술어(predicate)

술어(predicate)

- 일반적으로 관계(relation)를 정의함
- 하나 이상의 절(clause)로 구성됨

절(clause)은 다음 두 형태 중 하나

- 사실(fact)
 - ▶ 머리(head) 부분 다음에 `:-` 없이 `.`으로 끝나는 형태
 - ▶ 머리 부분이 나타내는 성질이 아무런 조건 없이 항상 참
- 규칙(rule)
 - ▶ 머리(head) 부분 다음 `:-` 와 몸체(body) 부분이 오고 `.`로 끝나는 형태
 - ▶ 머리 부분이 나타내는 성질이 특정 조건(즉, 몸체가 나타내는 성질)이 성립할 때만 참

```
%% 사실들만으로 정의된 술어 male
```

```
male(bob).
```

```
male(john).
```

```
%% 규칙 하나로만 정의된 술어 father
```

```
father(X) :- child(C,X), male(X).    % 자녀가 있고 남자인 조건이 성립해야 아버지로 인정
```

```
%% 사실과 규칙 각각 하나씩 2개의 절로 정의된 술어 is_list
```

```
is_list([]).                        % 사실(fact): []는 무조건 리스트로 인정
```

```
is_list([H|T]) :- is_list(T).      % 규칙(rule): [H|T]는 is_list(T)가 성립해야 리스트로 인정
```

사실(fact)을 똑같은 의미의 규칙(rule) 형태로 바꿔 쓰려면
머리 다음에 바로 `.`으로 끝내는 대신 뒤에 `:- true.`라고 쓰면 됨.

```
male(bob)  :- true.  % male(bob). 과 똑같은 의미  
male(john) :- true.  % male(john). 과 똑같은 의미
```

왜냐하면 `true`는 무조건 참으로 Prolog에서 미리 정의된 특별한 술어이기 때문.

Prolog의 규칙 형태의 절 $C :- P_1, P_2, \dots, P_n.$ 는

추론규칙 $\frac{P_1 \quad P_2 \quad \dots \quad P_n}{C}$ 혹은 논리식 $P_1 \wedge P_2 \wedge \dots \wedge P_n \implies C$ 에 해당한다.

(참고로, 논리식에서는 $\implies$ 대신 $\rightarrow$ 나 $\supset$ 기호를 쓰기도 함)

1.5. 항(term)과 일치화(unification)

1.5.1. 항(term)

Prolog에서 술어(predicate)가 뭔가 실행되어 움직이는 동적인 무언가라면, 항(term)은 정적인 데이터에 해당한다. 항(term)은 실행되는 것이 아님!!!

항(term)은 다음 셋 중 하나의 형태

- 변수(variable): `x`, `y1` 처럼 대문자로 시작. functor 위치에는 올 수 없음.
- 상수(constant)
 - atom: `a`, `b1` 처럼 소문자로 시작하거나 `'Hello World'` 처럼 따옴표로 둘러싸인 것
 - number: `123` 같은 정수나 `3.14` 같은 실수
- compound term / structure: `f(t1, ...)`
 - 여기서 `f` 는 함수자(functor) / 함수 기호(function symbol)
 - 여기서 `t1, ...` 등등은 항(term)

functor의 항수(arity)

- `f(t1,t2)` 에서 `f` 의 항수는 2
- `f(t1,t2,t3)` 에서 `f` 의 항수는 3

프로그램 코드에서는 같은 `f` 지만 Prolog 내부적으로 전자는 `f/2`, 후자는 `f/3` 와 같이 처리해 위의 두 `f` 를 서로 다른 functor로 확실히 구분함!!! (`g`, `h`, `f1`, `f4` 처럼 이름이 다른 경우나 마찬가지로)

1.5.2. 일치화(unification)

일치화(unification)란 두 항(term)을 서로 동일하게 만들 수 있는지 판단하는 과정이다. 그 과정에서 필요하다면 변수를 항으로 치환(substitute)함으로써 (혹은, 변수에 항을 대입함으로써) 두 항을 동일하게 만든다.

변수가 없는 경우에는 단순히 판단하기만 하면 됨

```
1  ?- a = 3 -> write(true), nl ; write(false), nl.  
2  ?- a = a -> write(true), nl ; write(false), nl.
```

 unification.pl

0: false

0: true

```
4  ?- f(1,3) = g(1,3) -> write(true), nl ; write(false), nl.  
5  ?- f(1,a) = f(1,3) -> write(true), nl ; write(false), nl.  
6  ?- f(1,3) = f(1,3) -> write(true), nl ; write(false), nl.
```

 unification.pl

0: false

0: false

0: true

변수에 무엇을 대입해도 일치화할 수 없는 경우들

```
8 ?- f(1,a) = f(X,3) -> format('success: X = ~w~n', [X])
9                        ; format("failure~n").
10 ?- f(1,Y) = g(X,3) -> format('success: X = ~w, Y = ~w~n', [X,Y])
11                        ; format("failure~n").
```

 unification.pl

0: failure

0: failure

변수를 적절히 치환하여 일치화에 성공하는 경우들

```
13 ?- f(1,3) = f(X,3) -> format('success: X = ~w~n', [X])
14                        ; format("failure~n").
15 ?- f(1,Y) = f(X,3) -> format('success: X = ~w, Y = ~w~n', [X,Y])
16                        ; format("failure~n").
```

 unification.pl

0: success: X = 1

0: success: X = 1, Y = 3

0:

살펴본 바와 같이 Prolog는 항(term)에 대한 일치화(unification) 알고리즘을 자체적으로 내장하고 있으므로, Prolog에서 제공하는 두 항의 일치화 기능을 위해 제공하는 특수한 2항 술어(predicate)인 `=`를 사용하기만 하면 됨.

주교재 0 Introduction

주교재에서 다루는 영역에 대한 개괄적 소개.

주교재에서 설명하면서 필요한 수학적 개념들 미리 정리.

주교재 0.1 오토마타, 계산가능성, 복잡도

계산이론(theory of computation)이 다루는 세 가지 영역

- 오토마톤 / 상태 기계: Automaton / State Machine
- 계산가능성: Computability - 어떤 문제가 계산으로 정의/해결 가능한지
- (계산)복잡도: (Computational) Complexity - 계산하기 위해 필요한 자원 소모 정도

참고로 오토마타(Automata)는 오토마톤(Automaton)의 복수형.

근데 한국어에서는 귀찮으니 그냥 1개도 오토마타라고 부르는 경우가 많음.

영어로 쓸 때는 주의해야 함.

계산이론의 이런 모든 분야를 관통하는 핵심 질문:

“What are the fundamental capabilities and limits of computation?”

“계산이라는 절차가 근본적인 역량과 한계는 무엇인가?”

즉, 계산이라는 것으로 어디까지는 할 수 있고 어떤 범위에서는 계산이라는 것이 불가능한지 탐구하는 것이 계산이론의 핵심이다.

(계산할 수 있을 거 같은데 ... 안되는 것도 있다는 이야기)

Complexity Theory 복잡도 이론

계산의 난이도를 다루는 이 분야의 핵심 질문:

“What makes some problems computationally hard and others easy?”

“어떤 문제들은 계산적으로 어렵고 다른 문제들은 쉬운 것은 무엇 때문인가?”

알고리즘 과목에서 많이 다루는 개념. (근데 일단 계산이 가능한 것들 중에서 ...)

Computability Theory 계산가능성 이론

이 분야의 선구자들: 쿠르트 괴델, 앨런 튜링, 알론조 처치

이 이론을 전개하기 위해서는 계산이 무엇인지 정의해야 함

계산의 모델(model)이라는 것을 정의하는 방식으로 계산이 무엇인지 정의함

Automata Theory 오토마타 이론

계산이 무엇인지 정의하는 한 방식으로 활용됨. 계산의 모델을 명확히 제시해야 계산이 가능한지, 가능하다면 그 계산이 얼마나 어려운지(필요 시간, 공간 등 자원) 판단 가능.

여러 모델들이 다른 분야에서도 응용됨

- 유한 오토마타 / 유한 상태 기계: Finite Automata / Finite State Machine
 - 텍스트 처리, 컴파일러에서 어휘 분석, 등등
- 푸시다운 오토마타 / 스택 오토마타: Pushdown Automata / Stack Automata
 - 언어의 문법 분석, 등등

주교재 0.2 수학적 표기법과 용어

집합 set

집합(set)은 원소/멤버(element/member)들의 모임.

집합의 원소나열법 표기: $S = \{7, 12, 57\}$

원소와 집합의 포함 관계 표현: $7 \in S, 8 \notin S$

원소나열법과 생략 기호를 활용한 표기:

- $\mathcal{N} = \{0, 1, 2, \dots\}$ (자연수의 집합)
- $\mathcal{Z} = \{\dots, -2, -1, 0, 1, 2, \dots\}$ (정수의 집합)

집합의 조건제시법 표기(set-builder notation): $S = \{m^2 \mid m \in \mathcal{N}\}$ (자연수의 집합)

공집합: $\emptyset = \{\}$ (원소가 하나도 없는 빈 집합)

원소가 딱 1개인 “한원소 집합”(singleton set). 예: $\{7\}$ (원소가 7 하나뿐인 집합)

원소가 딱 2개인 집합을 “순서 없는 쌍”(unordered pair) 또는 “집합쌍”이라고 함.

예: $\{7, 12\} = \{12, 7\}$ (원소가 7과 12뿐인 집합)

순서쌍(ordered pair, pair, 2-tuple). 예: $(7, 12) \neq (12, 7)$ (순서가 중요)

일반적으로 k -tuple. 예: $(7, 12, 5)$ (3-tuple)

집합의 연산

- 교집합(intersection): $A \cap B$
- 합집합(union): $A \cup B$
- 여집합(complement): $\bar{A} = \{x \mid x \in U, x \notin A\}$ 전체집합(U)이 정의되어 있어야 함
- 곱집합(cartesian product): $A \times B = \{(x, y) \mid x \in A, y \in B\}$
- 멱집합(power set): $\mathcal{P}(A) = 2^A = \{B \mid B \subseteq A\}$

집합 3개를 연속해서 곱집합(cartesian product)하면 3-tuple들의 집합이 만들어짐: $A \times B \times C = \{(x, y, z) \mid x \in A, y \in B, z \in C\}$

이런 식으로 정의되는 3-tuple에서는 $(a, b, c) = ((a, b), c) = (a, (b, c))$ 를 동치로 취급함. 즉, $(A \times B) \times C = A \times (B \times C) = A \times B \times C$ 라는 말. 하지만, 경우에 따라서는 이걸 구분해서 다루기도 하므로 주의가 필요. 맥락을 잘 살펴야.

벤다이어그램(Venn diagram)은 주교재 그림 참고

집합의 거듭제곱 표기

- $A^1 = A$
- $A^2 = A \times A$
- $A^3 = A \times A \times A$
- ...

일반적으로 $A^{k+1} = A \times A^k$

그런데 A^0 은 무엇이 되어야 할까? 일단, 공집합($\emptyset$)은 안됨!

유한집합의 경우 $A \times B$ 의 크기(원소의 개수)는 각각의 크기(원소의 개수)의 곱.
한쪽 크기가 0이면 곱집합의 크기도 0이 됨. 따라서, A^0 은 한원소 집합이어야 함.
근데, 그 한 원소를 뭘로 하지?

$A^0 = \{()\}$ (한원소 집합, 그 한 원소 $()$ 는 0-tuple)

$A = \{a_1, a_2, \dots\} = \{(a_1), (a_2), \dots\}$

원소 a_i 와 그 원소 하나만으로 이루어진 1-tuple (a_i) 는 동치로 취급.
또한 $((), a_i) = (a_i) = (a_i, ())$ 이것들도 모두 동치로 취급.

$A^0 \times A^1 = \{()\} \times \{a_1, a_2, \dots\} = \{(a_1), (a_2), \dots\}$

함수와 관계 functions and relations

함수(function)는 각 원소를 빠짐없이 정확히 하나씩의 원소에 일대일로 대응시키는 관계.
대응되는 대상 자체는 겹쳐도 됨. $x_1 \neq x_2$ 이더라도 $f(x_1) = f(x_2)$ 일 수 있음.

같은 x 를 서로 다른 원소에 대응시키면 안 됨. $f(x) = y_1 \neq y_2 = f(x)$ 이면 f 는 함수가 아님.

빠지는 게 있어도 안됨. $A = \{1, 2, 3\}$ 일 때, 함수 $f : A \rightarrow B$ 를 순서쌍의 집합으로 표현했을 때
 $f = \{(1, b_1), (3, b_3)\}$ 인 경우 함수가 아님. (원소 2가 대응 안됨)

이렇게 일대일 조건을 만족하면 부분함수(partial function). 이런 부분함수에 해당하는 구조를
데이터구조 이론이나 프로그래밍언어 라이브러리에서 딕셔너리(dictionary) 또는 맵(map)이라는
이름으로 부름.

빠짐없이 대응이 정의된 함수를 전함수(total function)라고도 부름. 그냥 함수라고 하면 전함수를 뜻함. 참고로 전함수도 부분함수임. 일대일 조건을 만족하는 게 부분함수의 정의이므로, 다른 조건을 좀더 만족한다고 해서 정의상 부분함수가 아니게 되는 것은 아니다.

함수의 리턴값이 진리값(참 또는 거짓)인 함수를 불리언 함수(boolean function)라고 부르지만 predicate(술어) 또는 proerty(성질)이라고도 부름.

그냥 집합 2개 A 와 B 사이의 관계(relation)는 $A \times B$ 의 부분집합이면 아무거나 됨.

함수와 관계의 항수(arity)

- k 항 함수(k -ary function) $f : A_1 \times A_2 \times \dots \times A_k \rightarrow B$ 에 넘겨주는 값들의 집합이 $A_1 \times A_2 \times \dots \times A_k$ 로 k 개의 곱집합. 마지막 공역(codomain) B 는 항수를 따질 때 포함 안됨.
- k 항 관계(k -ary relation) $R \subseteq A_1 \times A_2 \times \dots \times A_k$ 은 마지막까지 총 k 개 곱집합의 부분집합

k 항 함수는 (k 항 관계가 아니라) $k + 1$ 항 관계의 특별한 경우.

헛갈리지 않도록 주의!!! k 항 관계인지 검사하는 함수(즉, predicate / property)의 항수는 k 다. 주어진 k -tuple이 k 항 관계 $R \subseteq A_1 \times A_2 \times \dots \times A_k$ 를 만족하나 검사(즉, 진리값을 리턴)하는 함수 $p : A_1 \times A_2 \times \dots \times A_k \rightarrow \mathcal{B}$ 의 정의는 다음과 같다. (단, $\mathcal{B} = \{\text{TRUE}, \text{FALSE}\}$)

$$p(a_1, a_2, \dots, a_k) = \begin{cases} \text{TRUE} & (a_1, a_2, \dots, a_k) \in R \\ \text{FALSE} & (a_1, a_2, \dots, a_k) \notin R \end{cases}$$

관계의 표기법

관계 $R \subseteq A \times B$ 에서 일반적으로는

- $(a, b) \in R$ 이면 $R(a, b)$ 로 표기
- $(a, b) \notin R$ 이면 $\neg R(a, b)$ 로 표기

위의 표기법은 함수에 관계없이 활용 가능

특별히 함수(arity)가 2인, 즉 이항 관계(binary relation)일 때는
중위(infix) 표기법도 활용 가능함

- $(a, b) \in R$ 이면 $a R b$ 로 표기
- $(a, b) \notin R$ 이면 $\neg(a R b)$ 또는 $a \not R b$ 등으로 표기

그래프 Graph

그래프는 두 점(vertex)들을 선(edge)으로 연결하는 구조 $G = (V, E)$

- 무방향그래프 undirected graph
 - ▶ 보통 그냥 그래프 하면 일반적인 이론(이산수학)에서는 이거
 - ▶ 선들의 집합 E 를 순서없는쌍(unordered pair)으로 나타낼 수도 있고
 - ▶ 무방향그래프를 방향그래프로 나타낼 수 있음: 즉, $a \rightarrow b$ 와 $b \rightarrow a$ 한쌍을 항상 같이 그리기, 한쪽만 그리지 말고. 이렇게 할 때는 E 를 순서쌍의 집합으로 나타내되, E 가 대칭 관계(symmetric relation)가 되도록 구성해야
- 방향그래프 directed graph
 - ▶ 그림으로 그렸을 때 선이 화살표로 표시됨
 - ▶ 방향그래프는 무방향그래프와 달리 $E \subseteq V \times V$ 가 대칭 관계일 필요 없음.

그래프의 선에 뭔가 라벨/꼬리표(label)를 붙인 것을 Labeled Graph 라벨된 그래프

라벨이 없는 방향그래프가 $G = (V, \{(a, b), (b, c), \dots\})$ 이런 식이라면

라벨된 방향그래프는 이런 것 중 한 방식으로 표시 (edge를 triple, 즉 3-tuple에 해당하는 걸로)

- $G = (V, \{(a, l_1, b), (b, l_2, c), \dots\}) \iff$ 우리 교재에선 이거
- $G = (V, \{(a, b, l_1), (b, c, l_2), \dots\})$
- $G = (V, \{(l_1, a, b), (l_2, b, c), \dots\})$

우리 수업에서는 라벨된 방향그래프를 주로 다룸

문자열과 언어

알파벳 alphabet

- 문자열을 만들기 위한 기본 단위인 사용 가능한 글자(character) 혹은 심볼(symbol)의 집합
- 보통 Σ 로 표시 (나중에는 Γ 로 표시하는 것도 나오는데 그건 나중에)
- 일반적인 글자/심볼을 대표하는 변수는 $a, b, c, d \in \Sigma$ 처럼 같은 영문 알파벳 앞쪽 이탤릭체
- 구체적인 글자/심볼의 예시는 $0, 1 \in \Sigma_1$ 나 $b, c \in \Sigma_2$ 처럼 고정폭/타자기 폰트로

어떤 알파벳으로 이루어진 문자열 string over an alphabet

- 가능한 모든 문자열의 전체 집합 $\Sigma^* = \Sigma^0 \cup \Sigma^1 \cup \Sigma^2 \cup \dots$
- 일반적인 문자열을 대표하는 변수는 $x, y, z, w \in \Sigma^*$ 처럼 영문 알파벳 뒷쪽 이탤릭체

문자열은 곱집합의 원소이므로 원래는 tuple 표기법이지만 간략화된 문자열 표기법을 주로 활용.
영어 알파벳이라면 예를 들어 $abc = (a, b, c) \in \Sigma^3$ 이런 식으로 원래는 순서쌍이지만 문자열로 취급하는 경우 괄호와 쉼표 생략하고 표기.

문자열의 길이(length)는 문자열에 포함된 글자/심볼의 개수로 정의하며, x 의 길이를 $|x|$ 로 표기

길이 0개짜리 문자열(프로그래밍언어에서 보통 `""`)은 이런 이론에서는 ε 으로 표기

이어붙이기(concatenation 동사로는 concat) 표기

- 두 문자열 x 와 y 를 이어붙인 문자열을 xy 혹은 $x \cdot y$ 로 표기

(형식)언어 language

- (같은 알파벳을 기반으로 만든) 문자열의 집합. 즉, $L \subseteq \Sigma^*$

주교재 1 정규언어 Regular Languages

주교재 1.1 유한 오토마타 Finite Automata

Figure 1.1의 이해에 도움이 되는 Reddit에 올라온 참고 이미지

- <https://www.reddit.com/r/nostalgia/comments/gl8xws/>
- 문 열리고 지나가기 전에 (about to walk through) 밟는 패드/매트가 “front pad”
- 문 열린 쪽으로 나가서 (pass all the way through) 밟는 패드/매트가 “rear pad”

Figure 1.4는 상태전이도(state transition diagram, 줄여서 state diagram)의 예시

- state transition diagram은 방향그래프(directed graph)의 한 종류
- 라벨된 선/에지(edge)에 해당하는 화살표를 transition(상태전이)라고 부름
- 출발과 도착점이 같지만 라벨이 다른 상태전이 여러 개를 원래 각각 따로 그려야 하지만 하나로 겹쳐 모아 그리고 여러 라벨을 나열하여 표시하기도 함 (그림에서 0, 1로 라벨된 화살표)
- 그래프의 점(vertex)에 해당하는 상태(state)는 원으로 표시
- 상태들 중에서 하나의 시작/초기(start/initial) 상태를 허공에서부터 가리키는 화살표로 표시 (그림에서 q_1 . 언제나 딱 하나만 있어야)
- 상태들 중에서 최종/수락(final/accept state) 상태들은 테두리가 두 겹인 원으로 표시 (그림에서 q_2 . 일반적으로 여러 개 있거나 없을 수도)

유한 오토마타(Finite Automata, 줄여서 FA) 혹은

유한 상태 기계(Finite State Machine, 줄여서 FSM)는 여러 가지 방식으로 활용

- Figure 1.1-1.3처럼 FA의 각 상태마다 대응되는 실제 장치의 동작상태/모드
- 확률적 요소가 가미된 마르코프 체인(Markov Chain)은 통계적 시뮬레이션에 활용
- Figure 1.4처럼 시작부터 최종 상태까지의 전이 과정에 나타난 라벨(label)을 심볼로 취급하여 이를 이어붙인 문자열을 인식/처리하는 용도 $\Leftarrow$ 우리 주교재에서는 주로 이거에 초점

- 결정적 유한 오토마타(Deterministic Finite Automata, 줄여서 DFA)
 - 상태전이(transition)를 함수 $\delta : Q \times \Sigma \rightarrow Q$ 로 표현 가능
- 비결정적 유한 오토마타(Non-deterministic Finite Automata, 줄여서 NFA)도 존재
 - 상태전이(transition)를 위와 같은 형태의 함수로는 표현할 수 없는 경우까지 표현 가능한 관계 $\delta \subseteq Q \times \Sigma \times Q$ 로 표현

(결정적) 유한 오토마타(FA) M 의 수학적 정의(주교재 Definition 1.5)는

5-tuple(quintuple)인 $M = (Q, \Sigma, \delta, q_0, F)$ 로 표현

- Q 는 상태(state)들의 유한 집합
- Σ 는 알파벳 (심볼의 유한 집합)
- $\delta : Q \times \Sigma \rightarrow Q$ 는 상태전이(transition) 함수
- $q_0 \in Q$ 는 시작/초기(start/initial) 상태
- $F \subseteq Q$ 는 최종/수락(final/accept) 상태들의 집합

Figure 1.4, 1.6의 예시의 유한 오토마타 $M_1 = (\{q_1, q_2, q_3\}, \{0, 1\}, \delta_1, q_1, \{q_2\})$ 으로 정의한다면, 상태전이 함수 $\delta_1 : Q \times \Sigma \rightarrow Q$ 는 다음과 같다.

δ_1	0	1	
q_1	q_1	q_2	$\delta_1(q_1, 0) = q_1$ $\delta_1(q_1, 1) = q_2$
q_2	q_3	q_2	$\delta_1(q_2, 0) = q_3$ $\delta_1(q_2, 1) = q_2$
q_3	q_2	q_2	$\delta_1(q_3, 0) = q_2$ $\delta_1(q_3, 1) = q_2$

유한 오토마타/(상태)기계 “ M 이 문자열 w 를 수락(accept)/인식(recognize)한다”는 말은, 예컨대, $w = a_1a_2a_3$ 가 길이 3인 문자열이라면 $\delta(\delta(\delta(q_1, a_1), a_2), a_3) \in \{q_2\} = F_1$, 여기서 F_1 은 M 의 수락/종료(accept/final) 상태들의 집합. 이 예시를 차례차례 풀어 쓰자면

$$\delta(q_1, a_1) = q'$$

$$\delta(q', a_2) = q''$$

$$\delta(q'', a_3) = q''' \in \{q_2\} = F_1 \text{이면 } M \text{이 } w = a_1a_2a_3 \text{를 수락/인식한다고 판단.}$$

수학스러운 이론에서는 프로그래밍에서 있어보이듯 따로 용어까지 있는 함수/연산자 오버로딩 (overloading)이라는 개념이 숨쉬듯이 당연

그래서 교재에서는 글자 하나를 처리하는 $\delta : Q \times \Sigma \rightarrow Q$ 를 문자열에 나타나는 글자/심볼에 대해 연쇄적으로 호출하는 계산과정을 같은 이름의 $\delta : Q \times \Sigma^* \rightarrow Q$ 로 숨쉬듯 자연스럽게 오버로딩

$$\text{대략 } \delta(q, a_1 a_2 \dots a_{n-1} a_n) = \delta(\delta(\dots \delta(\delta(q, a_1), a_2), \dots), a_{n-1}), a_n)$$

증명에서 활용하는 수학적귀납법의 구조에 딱 떨어지게 맞도록 귀납적으로 정의하자면

$$\delta(q, \varepsilon) = q$$

$$\delta(q, aw) = \delta(\delta(q, a), w)$$

위의 귀납적 정의나 교재에 나오는 설명 및 수식에서 어느 것이 심볼/글자 하나를 처리하는 δ 이고, 어느 것이 여러 개의 (즉, 0개 이상의) 심볼로 이루어진 문자열을 처리하는 δ 인지 구분할 수 있어야

(형식)언어(language)를 “오토마타가 수락(accept)/인식(recognize)하는 문자열(string)”이라는 조건을 만족하는 집합으로 정의하기

$$L(M) = \{w \mid M \text{ accepts } w\}$$

오토마타 M 이 어떤 언어 $A \subseteq \Sigma^*$ 를 인식한다(즉, M accepts A)는 말은

- 언어에 속한 모든 문자열 $w \in A$ 은 수락하지만
- 그밖의 다른 모든 문자열 $x \notin A$ (또는 $x \in \bar{A}$)는 수락하지 않는다는 뜻.

즉, $L(M) = A$ 일 때 “ M 이 A 를 인식/수락”한다고 말한다.

(문자열에 대한 “인식/수락” 개념을 언어에 대한 것으로 확장. 일종의 개념적 오버로딩?)

정규 언어(regular language)란 어떤 (결정적) 유한 오토마타(DFA) M 으로 인식할 수 있는 언어.
앞으로 책에서 다음 세 방식으로 만들어지는 언어들이 정확히 똑같은 범주의 언어들이라는 것을 증명하는 내용으로 진행

1. DFA로 인식할 수 있는 언어
2. 유한집합을 모두 포함하며 다음 정규 연산들(교재 Definition 1.23: Regular Operations)에 대해 닫혀 있는 언어
 - $A \cup B$ 합집합(union)
 - AB 또는 $A \circ B$ 연결(concatenation). 이진 곱집합 $A \times B$ 와 마찬가지로
 - A^* 몇번이고 원하는 만큼 반복해 연결(Kleene star 또는 그냥 star)
3. 정규식으로 정의할 수 있는 언어 (이건 2번을 간결한 문법으로 표현한 것일 뿐)

참고로 다른 교재들에서는 2번이나 3번을 정규 언어의 정의로 약속하고 나머지가 똑같다는 식으로 책의 내용을 구성하기도 함.

우리는 당분간 1번의 언어가 2번에 나타나는 연산들에 대해 닫혀 있음을 보이는 데 집중할 예정.
근데 이게 뭘 증명하는지 큰 그림을 파악할 필요!!!!!!

- 1번(DFA로 인식 가능한 언어들)이 2번(정규연산에 닫혀있는 언어들)에 모두 포함 ???
- 2번(정규연산에 닫혀있는 언어들)이 1번(DFA로 인식 가능한 언어들)에 모두 포함 ???

교재 Theorem 1.25 정규 언어가 합집합에 대해 닫혀 있다는 정리

증명은 대략 다음과 같은 방향으로 진행

1. A 와 B 가 정규언어이면 정규언어의 정의에 따라

$A = L(M_1)$ 이고 $B = L(M_2)$ 인 DFA M_1 과 M_2 가 존재

2. $A \cup B = C$ 를 DFA 2개를 동시에 활용해서 인식하는 것은 다음과 같이 가능한데

- $s \in C$ 를 M_1 이 인식하는지 한번 돌려보고
- 같은 s 를 M_2 가 인식하는지 또 한 번 돌려봐서
- 둘 중 하나만 인식에 성공하면 합격!

그런데 문제는 이런 방식은 정규 언어의 정의와 정확히 맞지는 않음.

왜냐하면 정규언어란 정확히 1개의 DFA로 인식을 해야 되니까

3. 그러니까 방금 위에서 두 개의 기계로 하던 일을 똑같이 할 수 있는 하나의 DFA M 이 존재해서 이거 하나로 C 를 인식하는, 즉 $L(M) = C$ 가 되는 M 을 찾으면(만들어내면) 증명이 되는 것

구체적인 증명 방식, 즉 M 을 만들어내는 방식은 대략 이렇게

$$M_1 = (Q_1 = \{q_0, q_1, \dots\}, \Sigma, \delta_1, q_0, F_1)$$

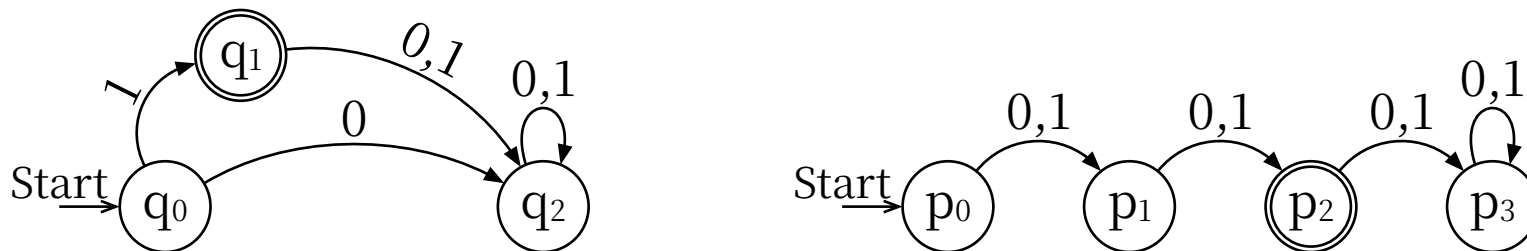
$$M_2 = (Q_2 = \{p_0, p_1, \dots\}, \Sigma, \delta_2, p_0, F_2) \text{로부터 다음을 만들어내기}$$

$$M = (Q = Q_1 \times Q_2, \Sigma, \delta, (q_0, p_0), F = \{(r_1, r_2) \mid r_1 \in F_1 \vee r_2 \in F_2\})$$

$$\text{여기서 } \delta((q, p), a) = (\delta_1(q, a), \delta_2(p, a))$$

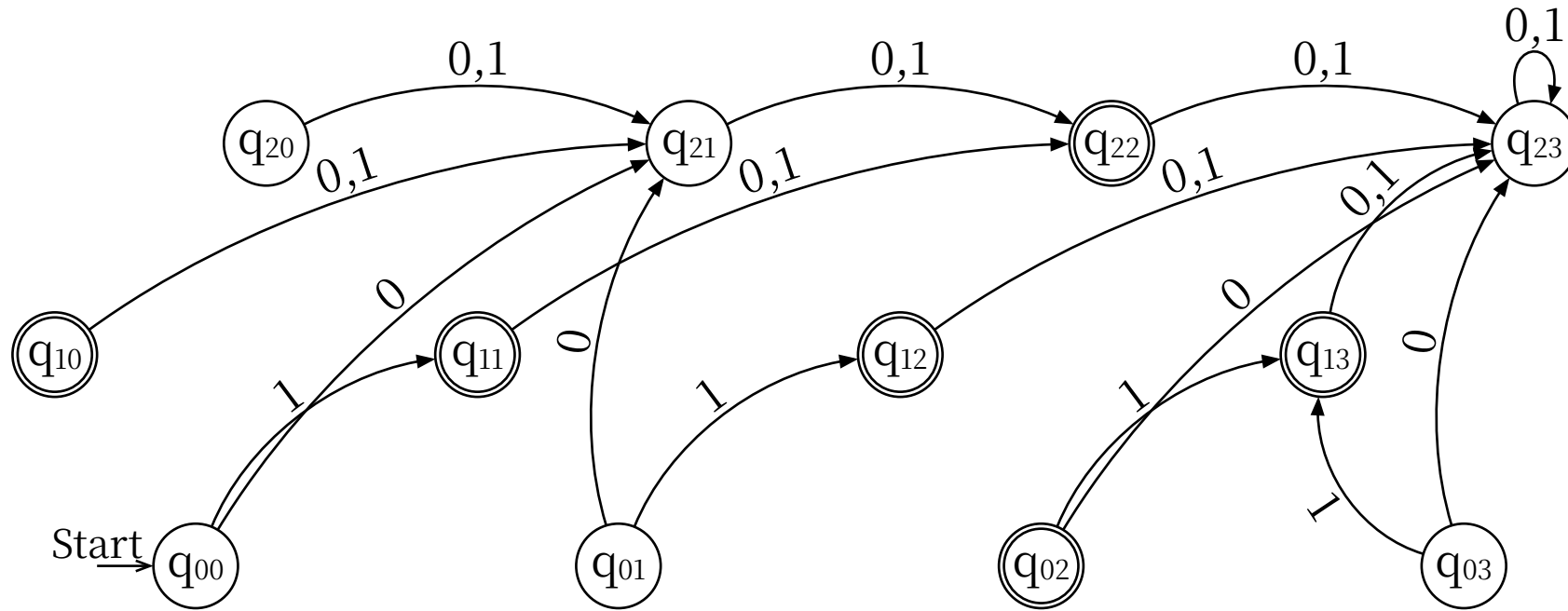
합집합 예시

알파벳 $\Sigma = \{0, 1\}$ 의 문자열 1만 수락하는 $M_1 = (Q_1 = \{q_0, q_1, q_2\}, \Sigma, \delta_1, q_0, F_1 = \{q_1\})$ 와 같은 알파벳의 길이 2인 문자열만 모두 수락하는 $M_2 = (Q_2 = \{p_0, p_1, p_2, p_3\}, \Sigma, \delta_2, p_0, F_2 = \{p_2\})$



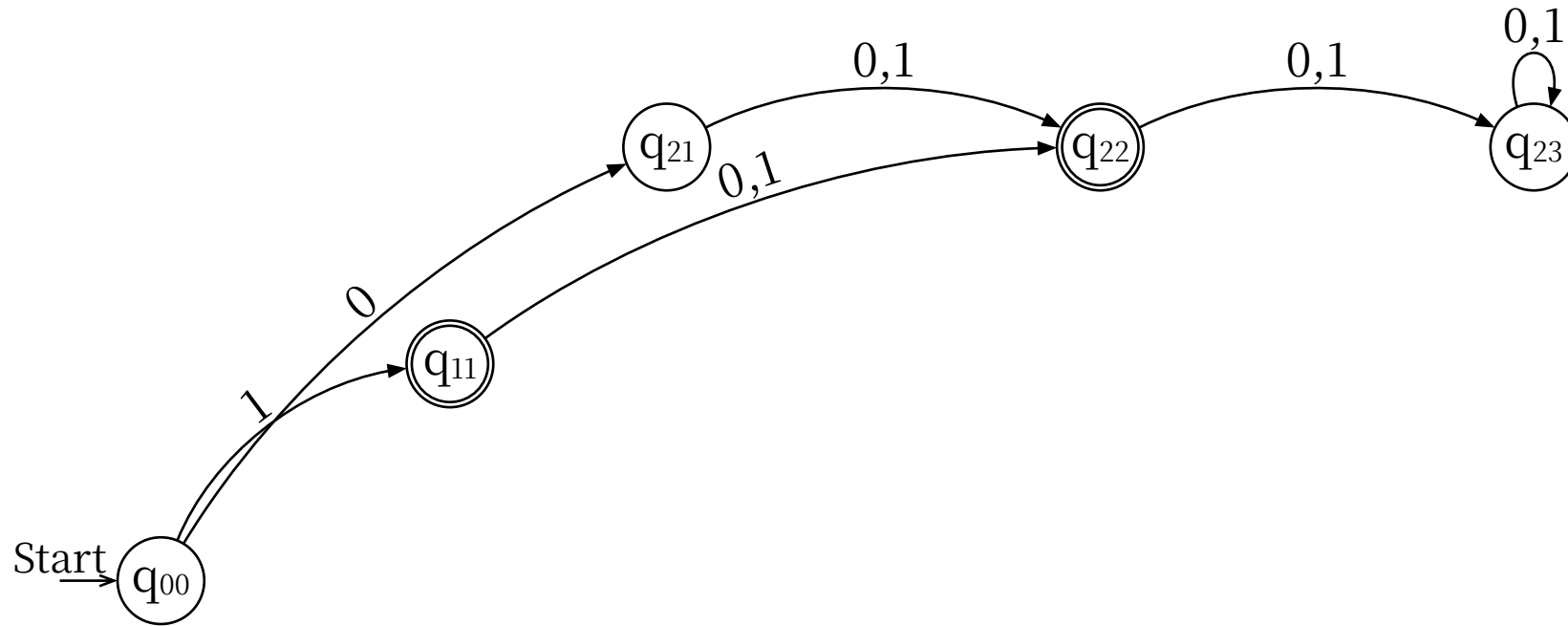
여기서 q_2 나 p_3 는 나머지 심볼들이 얼마만큼 더 들어오든 수락되지 못하도록 처리하는 상태인데, 이런 상태를 흡수/침몰/불능/죽은/함정/나락 상태(sink/dead/trap state)라고도 부르며 이를 강조하기 위해 q_d 나 q_d 처럼 아래첨자 d 를 써서 표기하기도 함

$L(M) = L(M_1) \cup L(M_2)$ 인 $M = (Q_1 \times Q_2, \Sigma, \delta, (q_0, p_0), F = F_1 \times Q_2 \cup Q_1 \times F_2)$ 를 그리면



근데 여기서 필요 없는 상태들이 있으니까 지워보자

필요 없는(즉, 시작 상태에서 도달 불가능 unreachable) 상태 제거 연습



같은 방식(즉, DFA만)으로 연결(concatenation)에 대해 닫혀있음을 보이기 어려운 이유
대략 이런 방향으로 진행해야

1. A 와 B 가 정규언어라면 정의에 따라 $L(M_1) = A$ 이고 $L(M_2) = B$ 인 DFA M_1 과 M_2 가 존재
2. 어떤 문자열 w 를 잘 분해해서(모든 가능한 경우 중 하나라도 되는 게 있으면 ok) $w = xy$ 로
 - 앞부분 x 는 M_1 한테 넘겨서 뽀 돌려서 accept 되나 확인하고 된다면 다음으로 넘어가서
 - 뒷부분 y 는 M_2 한테 넘기서 뽀 돌려서 accept 되는지 확인

이렇게 되면 $AB = C$ 에 w 가 속한다고 이런 식으로 DFA 2개를 활용해서 판단 가능

3. 문자열 w 를 나눌 때 최대 개수가 한정된 유한한 최대범위 내의 경우의 수 이내로만 따질 수 있다면, 이것 혼자 처리하는 한개짜리 DFA M 을 만들 수 있을 거 같기는 한데 ...

모든 가능한 경우를 고려한다는 게 최대치가 얼마인지 정하기가 어려움

- A 와 B 가 둘다 유한집합이면 쉽게 모든 가능한 경우의 최대 분기 개수를 한정할 수 있음
- 문제는 A 와 B 가 무한집합일 수도 있다는 점

즉, 2번 과정에서 w 의 길이가 길어지면 길어지수록 고려해야 할 경우의 수가 그 길이에 비례해서 많아지므로 어디까지를 경우의 수 최대값 한도로 정하기 어려움.

경우의 수 최대값 한도가 정해지지 않으면 M 의 상태 집합을 최대 몇개로 잡아야 할지 ...

유한오토마타는 상태 집합이 유한집합이어야 하는데 바로 그게 문제

주교재 1.2 비결정성 Non-determinism

일반적인 의미로는 비결정적(non-deterministic)이라고 하면 결정적인 경우를 제외한 나머지

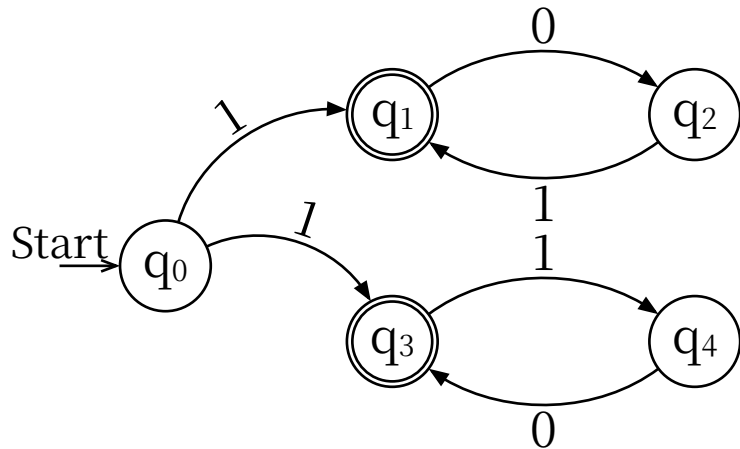
하지만 “비결정적 유한오토마타”(NFA)는 “결정적 유한오토마타”(DFA)의 개념을 사실상 포함

- NFA: $(Q, \Sigma, \delta, q_0, F)$ 에서 δ 가 전이“관계” transition relation
- DFA: $(Q, \Sigma, \delta, q_0, F)$ 에서 δ 가 전이“함수” transition function

함수(function)은 관계(relation)의 일종(함수 = 전단사관계; one-to-one 단사, onto 전사)

그러니까, 모든 DFA는 NFA 형태로 쉽게 옮겨적을 수 있음. 함수를 관계로 표현하기만 하면 됨.

NFA 예시: 첫 글자 1이 나온 다음에, 01이 (0회 이상) 반복 또는 10이 (0회 이상) 반복



화살표에 대응되어 처리되지 않는 입력은 대략 DFA의 흡수/죽음/함정/나락 상태(sink/dead/trap state)에 해당

여러 선택 가능한 경로들 중 하나라도 수락/최종 상태(accept/final state)에 도달하면 됨

<https://static0.cbrimages.com/wordpress/wp-content/uploads/2019/05/doctor-strange-avengers-infinity-war.jpg>

“어벤저스: 인피니티워” 영화 중에서, 1400만개의 미래 중 타노스를 막을 수 있는 단 1개라도 ...

교재 Figure 1.28, Figure 1.29 참고

δ 가 관계(relation)인 것 말고 또 다른 NFA의 추가 기능: ϵ 전이(ϵ -transition)

- 그림에서는 화살표 라벨에(label)에 ϵ 표기
 - 엡실론 생각하지 않으면 $\delta \subseteq Q \times \Sigma \times Q$ 일텐데
 - 엡실론 전이를 허용하면 $\delta \subseteq Q \times (\Sigma \cup \{\epsilon\}) \times Q$
 - 원래 오토마타는 글자/심볼 하나를 보고/처리하고 화살표를 따라 수락하지만
 - ϵ 이 붙어있는 화살표는 글자/심볼 처리하지 않고 그대로 두고 화살표를 따라 이동 허용
 - 활용 방식
 - ▶ 출발/도착이 같은 다른 글자/심볼과 함께 쓰면 (교재 Figure 1.27)
그 글자가 있어도 되고 없어도 된다는
 - ▶ 수락/종료(accept/final) 상태를 항상 1개로 만들 수 있음
- 원래 NFA의 수락상태들 일반상태로 만들고 거기서 새로운 하나의 수락상태로 엡실론 전이 추가
- 다른 교재나 자료들에서는 엡실론 전이를 점선 화살표로 강조하기도 함

이런 엡실론 전이를 포함한 NFA가 인식하는 언어는
연결(concatenation)에 대해 닫혀있음을 보이기/증명하기 쉬워짐

근데 NFA는 DFA가 아니잖아? ...

NFA가 할 수 있는 일이 많아 보여도 사실 DFA가 할 수 있는 범위의 일밖에 못한다 ...
그러니까 뽀뽀이다 $\leq$ 이것까지 증명하면

DFA가 인식하는 언어(즉, 정규언어)가 연결에 대해 닫혀있다는 것을 증명하는 셈

주교재 1.3 정규식 Regular Expressions

주교재 1.3 정규언어가 아닌 언어

다음 장